

Записки по ДАА. Упр 1: Въведение в алгоритмите, Big O notation

Цел на курса по Дизайн и анализ на алгоритми

Целта на курса е да изгради умения за систематично решаване на изчислителни проблеми чрез ефективни алгоритми. Стремим се да усвоим основни алгоритмични парадигми и да разбираме кога и защо да ги прилагаме. В курса анализираме коректността и сложността на алгоритмите, за да оценяваме тяхната ефективност. Основният фокус е върху развиване на алгоритмично мислене и умения за аргументиран избор на решения.

Какво са алгоритмите?

Алгоритъм $\Leftrightarrow$ дума която програмистите използват когато не могат да обяснят какво прави кода им.

Най-общо казано, **алгоритъм** е ясно дефинирана изчислителна процедура, която приема някакви входни данни (една или повече стойности) и произвежда изходни данни (една или повече стойности). Тоест алгоритъмът представлява последователност от краен брой стъпки, които преобразуват Входа в изход.

Например, често се налага да сортираме поредица от числа във възходящ ред. Това е много често срещан проблем на практика и е отличен пример за въвеждане на стандартни техники за проектиране и анализ на алгоритми. Формално проблемът за сортиране се дефинира така:

Вход: Поредица от n числа $\langle a_1, a_2, \dots, a_n \rangle$

Изход: Пермутация (пренареждане) $\langle a'_1, a'_2, \dots, a'_n \rangle$ на входната поредица, така че

$$a'_1 \leq a'_2 \leq \dots \leq a'_n.$$

Например, ако входът е $\langle 31, 41, 59, 26, 41, 58 \rangle$, алгоритъмът за сортиране ще върне като изход $\langle 26, 31, 41, 41, 58, 59 \rangle$.

Тъй като много програми използват сортирането като междинна стъпка, то е една от фундаменталните операции в компютърните науки. Затова съществуват много добри алгоритми за сортиране. Кой от тях е най-подходящ зависи от различни фактори, като например: броя на елементите,

доколко те вече са частично подредени, възможните ограничения върху стойностите им, архитектурата на компютъра и типа използвана памет – оперативна памет, дискове или дори магнитни ленти.

Казваме, че един алгоритъм е **коректен**, ако за всеки възможен вход той завършва работа и дава правилен резултат. Такъв алгоритъм се счита, че решава съответния проблем. Некоректен алгоритъм може никога да не приключи за някои входове или да приключи с грешен резултат.

Изчислителен модел: RAM (Random Access Machine)

За да можем коректно да анализираме ефективността на алгоритмите, е необходимо да работим с абстрактен модел на изчисление. Този модел описва как алгоритъмът „вижда“ паметта, какви операции може да извършва и как измерваме цената на тези операции.

В курса най-често използваме **RAM модела** (*Random Access Machine*), който е опростен, но достатъчно близък до реалните компютри.

Основна идея на RAM модела

RAM моделът е абстракция на съвременен компютър със следните ключови характеристики:

- **Произволен достъп до паметта** – всяка клетка в паметта може да бъде достъпена за еднакво време, независимо от адреса ѝ.
- **Последователно изпълнение на инструкции** – програмата се изпълнява ред по ред, с възможност за условни и безусловни преходи.
- **Неограничен размер на паметта** – моделът допуска неограничен брой клетки памет, което улеснява теоретичния анализ.

Целта на този модел **не е да симулира хардуера в детайли**, а да ни позволи ясно и еднозначно да измерваме сложността на алгоритмите.

Памет и регистри

RAM машината разполага с два основни типа памет:

- **Краен брой регистри**

Регистрите се означават с

$$r_0, r_1, \dots, r_n, \quad n \in \mathbb{N}^+$$

Един от тях (обикновено r_0) играе ролята на **аккумулятор** – в него се записват резултатите от аритметичните операции.

- **Основна памет**

Състои се от безкрайна редица клетки, номерирани с $0, 1, 2, \dots$

Всяка клетка може да съдържа цяло число с произволна големина.

Важно е да се подчертае, че в RAM модела **стойностите могат да бъдат произволно големи**, което е нереалистично от хардуерна гледна точка, но удобно за теоретичен анализ.

Програма и инструкции

Програмата в RAM модела е крайна последователност от прости инструкции. Основните типове инструкции са:

1. Достъп до паметта

- присвояване между регистри и памет
- четене и запис в паметта чрез адрес

2. Управление на изпълнението

- безусловен преход (goto)
- условен преход (if ... then goto ...)

3. Аритметични операции

- събиране, изваждане, умножение
- целочислено деление и остатък

4. Операции с индексни регистри

- увеличаване или намаляване на стойността на регистър

Тези инструкции са достатъчни, за да опишем всички алгоритми, които ни интересуват в курса.

Цена на изпълнение на операциите

За да анализираме времевата сложност на алгоритмите, трябва да дефинираме **цената на елементарните операции**. В RAM модела най-често се използват два подхода:

1. UCM (Unit Cost Measure)

При този модел **всяка елементарна операция струва 1**, независимо от размера на числата, върху които работи.

Това е удобен и интуитивен модел, който:

- се използва в повечето класически анализи
- е подходящ, когато входните числа не нарастват прекалено бързо

2. LCM (Logarithmic Cost Measure)

Тук цената на операцията зависи от **размера на числата**, които участват в нея.

Например, операция с число x има цена пропорционална на $S(x)$, където $S(x)$ е размерът на представянето му.

Този модел е по-реалистичен, но и по-сложен, затова в началните анализи често се използва UCM.

В рамките на курса основно ще работим с UCM, освен ако изрично не е указано друго.

Представяне на числата в паметта

Числата в RAM модела се записват в фиксирана бройна система с основа k . Една клетка памет може да съхранява ограничено количество информация, което означава, че големите числа могат да заемат повече от една клетка.

За целите на анализа обаче **рядко навлизаме в тези детайли**, освен когато изрично анализираме влиянието на размера на числата върху сложността.

Какво е важно да запомним?

- RAM моделът е **абстракция**, не реален компютър.
- Използва се, за да:
 - сравняваме алгоритми
 - анализираме времева и пространствена сложност
- Игнорира хардуерни детайли като кешове, паралелизъм и тактова честота.
- Дава ни ясен и формален начин да броим „колко работа“ извършва алгоритъмът.

Преди да дадем формална дефиниция ще започнем с няколко примера за да започнем да придобиваме интуиция към това как измерва сложността на един алгоритъм.

Пример 1:

Играем игра срещу противник. В играта нашият противник си намисля число от 1 до N а ние трябва да го познаем, като всеки път когато направим предположение противника ни казва дали числото което сме предположили е по-малко или по-голямо от намисленото от него число.

1 2 3 4 5 6 7 8 9 10 11 12 13 $N-2$ $N-1$ N

Лоша стратегия (интуитивна, но неефективна)

Играчът пита:

- „По-голямо ли е от 1?“
- „По-голямо ли е от 2?“
- „По-голямо ли е от 3?“
- ...

В най-лошия случай ще са нужни **n въпроса**.

Добра стратегия

Играчът играе „умно“:

1. Пита:
„По-голямо ли е от $n/2$?“
2. Ако отговорът е „да“ → числото е в интервала $(n/2, n]$
Ако е „не“ → числото е в интервала $[1, n/2]$
3. После пита средата на останалия интервал.
4. Повтаря, докато намери числото.

Пример 2:

Нека си представим че сме си изпразли чорапите и след пране искаме да ги подредим в шкафчето ни. Искаме да подредим нашите чорапи по чифт.

=====

Първи вариант да го направим:

1. Взимаме чорап от купчината
2. Поглеждаме чорапа
3. Слагаме го на правилното място

Ако чорапите ни са:

1. 10 => гледаме 10 пъти
2. 100 => гледаме 100 пъти
3. 1000 => гледаме 1000 пъти

Тоест работата която трябва да свършим расте пропорционално на броя чорапи.

=====

Втори вариант да го направим:

1. Взимаме чорап от купчината
2. За съответния чорап гледаме всички останали чорапи за да видим дали има чифт
3. Поставаме го на правилното място

Ако чорапите ни са:

1. 10 => гледаме ~ 100 пъти
2. 100 => гледаме ~ 10000 пъти
3. 1000 => гледаме ~ 1000000 пъти

В началото видяхме малко увеличение в работата която трябва да свършим, но когато броят чорапи се увеличи, работата която трябва да свършим нарасна значително. По общо за n чорапа трябва да направим $\sim n^2$ стъпки за да си свършим работата.

Определение 1. (Асимптотично неотрицателна функция)

Нека f е реалнозначна функция, дефинирана върху множеството на реалните или целите положителни числа.

Функцията f се нарича **асимптотично неотрицателна**, ако

$$\exists n_0 \in \mathbb{N} \forall n \geq n_0: f(n) \geq 0.$$

Ако неравенството е строго, т.е. $f(n) > 0$, функцията се нарича **асимптотично положителна**.

Определение 2. (Асимптотично сравнение — $\leq$)

Нека f и g са асимптотично неотрицателни функции. Казваме, че f е **асимптотично не по-голяма** от g и записваме

$$f \leq g$$

ако

$$\exists c \in \mathbb{R}^+, \exists N \in \mathbb{N} \forall n > N: f(n) \leq c \cdot g(n).$$

Определение 3. (Big O)

Класът на **асимптотично не по-бързо растящите функции** от g е множеството

$$O(g) = \{f \mid f \leq g\}.$$

$O(g)$ съдържа всички функции, които растат асимптотично **НЕ по-бързо** от g .

Записваме:

$$f \in O(g), \quad f = O(g) \text{ или } f \leq g.$$

Определение 4. (Big Ω)

Класът на **асимптотично не по-бавно растящите функции** от g е множеството

$$\Omega(g) = \{f \mid g \leq f\}.$$

$\Omega(g)$ съдържа всички функции, които растат асимптотично **НЕ по-бавно** от g .

Записваме:

$$f \in \Omega(g), \quad f = \Omega(g) \text{ или } f \geq g.$$

Определение 5. (small o)

Класът на **асимптотично по-бавно растящите функции** от g е множеството

$$o(g) = \{f \mid \forall c > 0 \exists N \in \mathbb{N} \forall n > N: f(n) < c \cdot g(n)\}.$$

$o(g)$ съдържа всички функции, които растат асимптотично **по-бавно** от g .

Записваме:

$$f \in o(g), \quad f = o(g) \text{ или } f < g.$$

Определение 6. (small ω)

Класът на **асимптотично по-бързо растящите функции** от g е множеството

$$\omega(g) = \{f \mid \forall c > 0 \exists N \in \mathbb{N} \forall n > N: c \cdot g(n) < f(n)\}.$$

$\omega(g)$ съдържа всички функции, които растат асимптотично **по-бързо** от g .

Записваме:

$$f \in \omega(g), f = \omega(g) \text{ или } f \succ g.$$

Определение 7. (Theta нотация — Θ)

Класът на **асимптотично еднаво растящите функции** с g е множеството

$$\Theta(g) = O(g) \cap \Omega(g).$$

Еквивалентно:

$$\Theta(g) = \{ f \mid \exists c_1, c_2 > 0, \exists N \in \mathbb{N} \forall n > N: c_1 g(n) \leq f(n) \leq c_2 g(n) \}.$$

$\Theta(g)$ съдържа всички функции, които растат асимптотично **еднаво** с g .

Записваме:

$$f \in \Theta(g), f = \Theta(g) \text{ или } f \asymp g.$$

**За целите на нашият курс, ще използваме за определяне
времевата сложност на алгоритмите само Big O нотацията.**

RAM модел и Big-O анализ на алгоритми

Основната цел на RAM модела в курса е **да осигури формална основа за Big-O анализа**.

Big-O нотацията описва **как нараства времето (броят стъпки) за изпълнение на алгоритъм**, когато нараства размерът на входа.

RAM моделът отговаря на въпроса: **как точно броим „времето“**.

Как RAM моделът дефинира „време“

В RAM модела:

- Алгоритъмът е последователност от **елементарни операции**
- Всяка елементарна операция има **цена**

- **Времето за изпълнение** на алгоритъм е сумата от цените на всички изпълнени операции

При **Unit Cost Measure (UCM)** приемаме:

Всяка елементарна операция струва 1.

Това означава, че времето за изпълнение е просто **броят на изпълнените инструкции**.

Следните операции върху числа могат да се изпълняват за константно време в RAM модела, при условие, че операндите се побират в една машинна дума:

- $a + b, -a, a * b, a/b (b \neq 0)$
- $a^b, \lfloor \log_b a \rfloor, a!$ (в случаите, когато $a!$ се побира в машинната дума)
- $a \div b$ (целочислено), $a \bmod b$
- $a = b, a < b$ (може да считаме, че това са операции, чийто резултат е 0 или 1)
- $a \vee b, \neg a, a \gg b, a \ll b$

Разбира се, това не са всички операции. Има доста такива, които могат да се изразят като суперпозиция на изложените (например $a \leq b = a < b \vee a = b$).

Какво всъщност прави Big O?

Както споменах по-горе, Big O пита само един въпрос:

Как нараства количеството работа, когато нараства размерът на входа?

И игнорира неща като:

- колко бърз е компютърът
- дали кодът е писан на C++ или Python
- дали операцията отнема 1 ms или 5 ms

Защото тези неща се менят, но начинът по който расте работата не променя.

Свойства

Свойство 1. (Транзитивност)

Ако $f \sigma g$ и $g \sigma h$, то $f \sigma h$, за всяко $\sigma \in \{\leq, \geq, <, >, \asymp\}$.

Свойство 2. (Рефлексивност на $\mathcal{O}$, Ω , Θ)

Релациите $\leq$, $\geq$ и $\asymp$ са рефлексивни:

$$f \sigma f \text{ за всяко } \sigma \in \{\leq, \geq, \asymp\}.$$

Свойство 3. (Антисиметричност)

$$f \geq g \text{ и } g \geq f \Leftrightarrow f \asymp g.$$

Свойство 4. (Симетричност на Θ)

$$f \asymp g \Leftrightarrow g \asymp f.$$

Доказателство.

Дефиницията на Θ е симетрична:

$$c_1 g(n) \leq f(n) \leq c_2 g(n) \Leftrightarrow \frac{1}{c_2} f(n) \leq g(n) \leq \frac{1}{c_1} f(n).$$

Свойство 5. (Обръщане на неравенствата)

$$f \leq g \Leftrightarrow g \geq f, \quad f < g \Leftrightarrow g > f.$$

Свойство 6. (Суми и множители)

а) $\max \{f, g\} \asymp f + g$

Доказателство.

За достатъчно големи n :

$$\frac{1}{2}(f(n) + g(n)) \leq \max \{f(n), g(n)\} \leq f(n) + g(n).$$

Следователно двете функции са от един и същи асимптотичен порядък.

б) Ограничен множител не влияе на растежа

Ако

$$\exists c_1, c_2 > 0: c_1 \leq f(n) \leq c_2,$$

то

$$g \cdot f \asymp g.$$

Свойство 7. (Граници и o, ω)

Нека f и g са асимптотично положителни.

а)

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0 \Leftrightarrow f = o(g).$$

Доказателство.

По дефиниция на граница:

$$\forall \varepsilon > 0 \exists N: \frac{f(n)}{g(n)} < \varepsilon \Leftrightarrow f(n) < \varepsilon g(n),$$

което е точно дефиницията на $o(g)$.

б)

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty \Leftrightarrow f = \omega(g).$$

Доказателство.

Следва като реципрочно твърдение на предходното.

Свойство 8. (Граница и Θ)

Ако

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = c > 0,$$

то

$$f \asymp g.$$

Доказателство.

Нека $0 < \varepsilon < c$. За достатъчно големи n :

$$\begin{aligned} c - \varepsilon &\leq \frac{f(n)}{g(n)} \leq c + \varepsilon \\ \Rightarrow (c - \varepsilon)g(n) &\leq f(n) \leq (c + \varepsilon)g(n), \end{aligned}$$

което по дефиниция означава $f \in \Theta(g)$.

Забележка (обратното не е вярно)

Например:

$$f(n) = (2 + \sin n) n, g(n) = n.$$

Имаме $f \asymp g$, но

$$\frac{f(n)}{g(n)} = 2 + \sin n$$

няма граница.

